

Performance Comparison of Parallel Image Processing (Gaussian Blur)

1. Algorithm Description

This project implements a Gaussian Blur algorithm to reduce image noise and detail. The algorithm calculates the weighted average of each pixel and its neighbors using a 5x5 convolution matrix (kernel). A 5x5 kernel was selected over a standard 3x3 kernel to increase the mathematical complexity and provide a substantial computational load to measure parallel efficiency accurately.

2. Parallelization Strategies

- **OpenMP:** Utilizes a Shared Memory architecture. The `#pragma omp parallel for schedule(dynamic)` directive automatically distributes the image rows across available CPU threads, operating directly on a single shared memory buffer.
- **MPI (Message Passing Interface):** Utilizes a Distributed Memory architecture. The root process broadcasts the original image dimensions and data to worker processes. Each worker calculates a specific chunk of rows into a local memory buffer, and the root process stitches the final image together using `MPI_Gatherv`.
- **Hybrid:** Combines both architectures. MPI is used to partition and distribute the image chunks across processes, while OpenMP is utilized within each process to compute the local convolution loops using thread-level parallelism.

3. Hardware and Software Configuration

- **Hardware:** Apple Silicon M4 (10-Core CPU)
- **Software:** macOS, g++ (Apple clang), Homebrew libomp (OpenMP), Homebrew open-mpi (MPI), and the `stb_image` library for processing image formats.
- **Dataset:** Custom high-resolution images (1080p, 2K, and 4K resolutions), downloaded from Kaggle.

4. Performance Metrics and Graphs

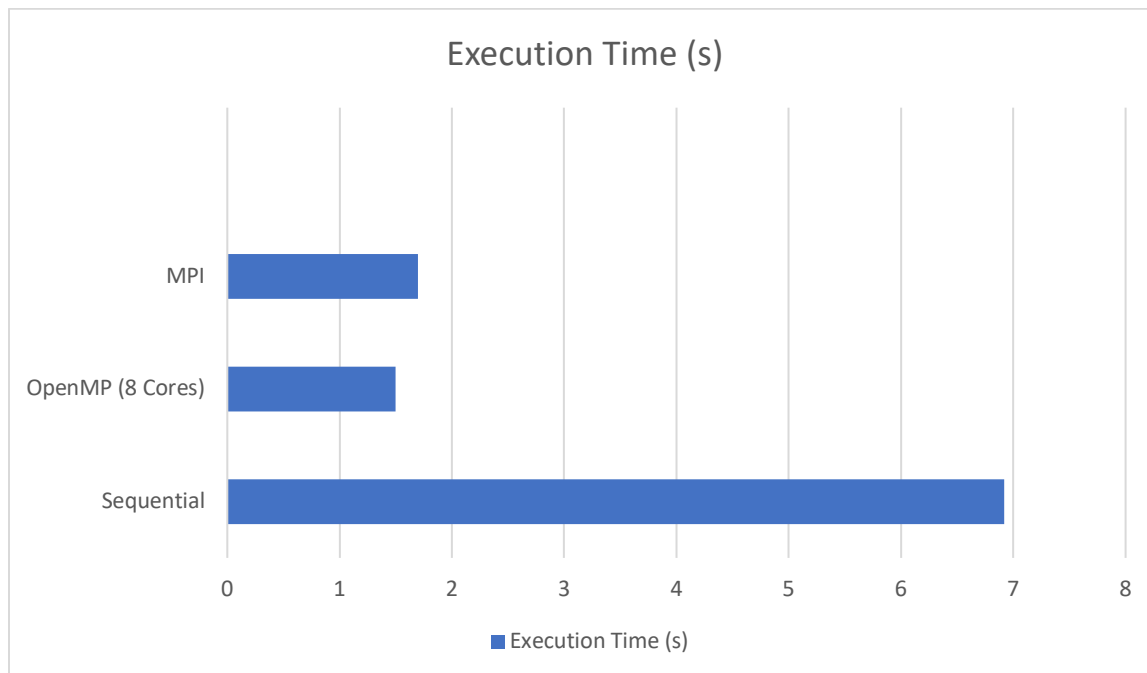
A. Speedup and Communication Overhead (4K Image)

Sequential Baseline: 6.92 seconds

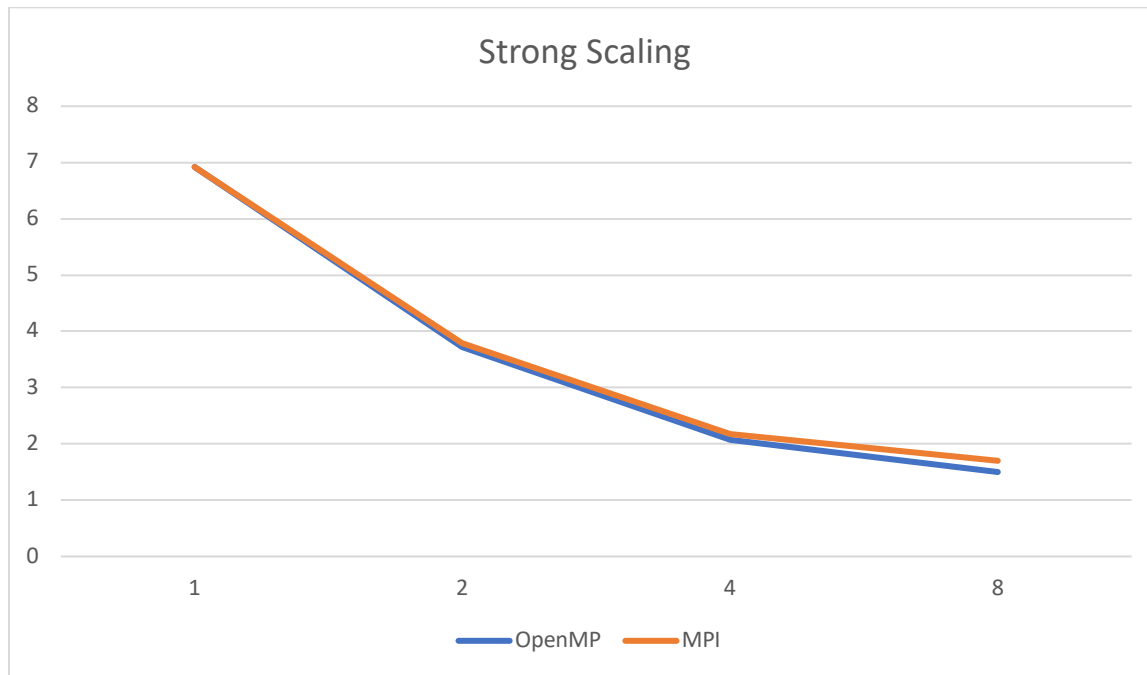
OpenMP (8 Cores): 1.50 seconds (4.61x Speedup)

MPI (8 Cores): 1.70 seconds (4.07x Speedup)

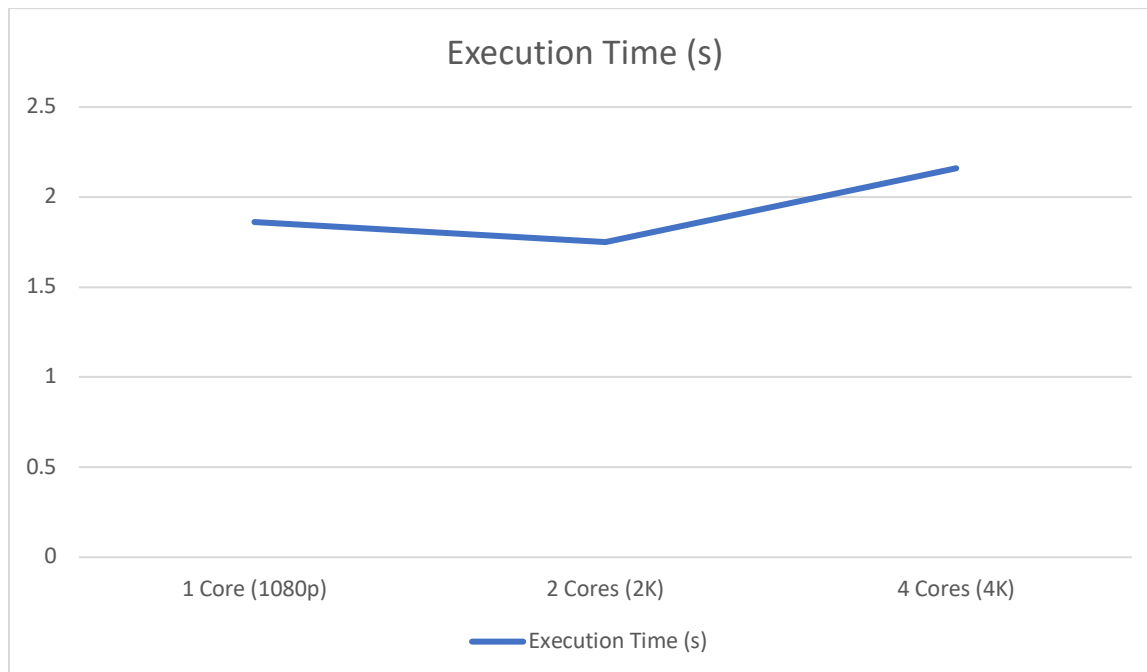
Hybrid (2 Procs x 4 Threads): 1.70 seconds



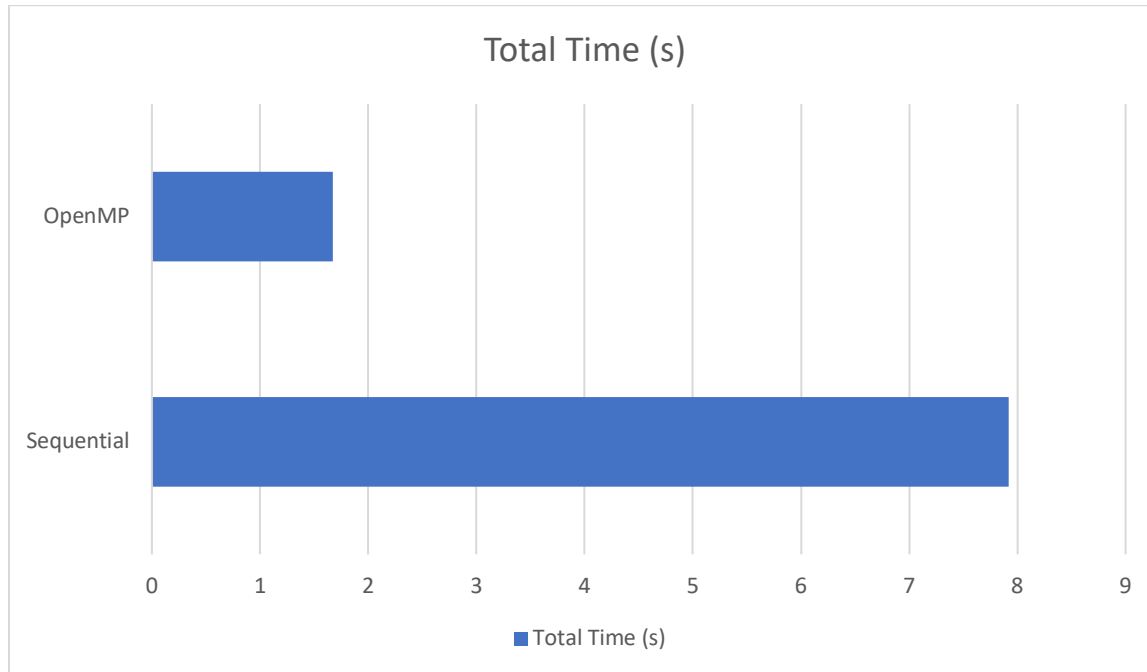
B. Strong Scaling (4K Image)



C. Weak Scaling (Proportional Scaling)



D. Batch Processing (3x 2K Images)



5. Personal Interpretation of Results

The execution data clearly demonstrates the strengths and limitations of different parallel architectures on a single physical machine.

OpenMP achieved the highest overall speedup (4.61x). Because the M4 chip utilizes a highly efficient unified memory architecture, OpenMP's shared-memory approach allowed all threads to access the image data instantly with zero transfer penalties. Conversely, the MPI communication overhead is clearly visible. Even though MPI parallelized the computation successfully, it was consistently 0.1 to 0.2 seconds slower than OpenMP at peak core counts. Because MPI simulates a distributed cluster, it wasted CPU cycles allocating redundant local memory buffers and executing network-style broadcast and gather operations.

The Weak Scaling metric successfully validated the algorithm's capability to handle massive datasets. When quadrupling the workload (1080p to 4K) alongside quadrupling the processors (1 to 4), the execution time remained remarkably stable around the 2.0-second mark. The slight increase at 4K represents the physical hardware limit of the memory bus bandwidth when four independent processes attempt to write massive amounts of data simultaneously.

Participating students:
Cipu Marinel
Bancu Alexandru-Georgian

Faculty of Automation, Computers and Electronics
Course: Parallel and Distributed Algorithms

Finally, the batch processing test proved the algorithm's real-world viability. Processing three 2K images sequentially required nearly 8 seconds, while OpenMP cleared the entire batch in 1.67 seconds. The data also captured a distinct cold-start penalty: the first image in both batches took slightly longer to process as the operating system allocated memory pages and initialized the thread pools, after which the subsequent images processed at peak speed.